

Android App for Commercial Detection

<https://gemini.google.com/app/9cad3b3351564618>

User prompt: Please suggest possible implementation of an android app that detects when the device's audio comes from commercials - and lower the volume to minimum until the commercials end. It should not be deployed to Google App Store - only for personal usage, so privileges should not be an issue. It should be runnable locally on the phones hardware. I imagined using some existing light neural network for audio processing, with at most some fine tuning, or maybe logistic regression that is applied to one of it's last layers. Tell me what you think. You should not implement it now - only offer possible implementations.

Response: This is a fantastic concept for a personal utility app. Because you are sideloading this for personal use and don't have to navigate Google Play's strict policy restrictions regarding background audio capture or screen reading, you have a lot of architectural freedom.

Your intuition about using a pre-trained network and applying logistic regression to its embeddings is spot-on for on-device edge AI.

Here is a breakdown of possible implementations and the system architecture required to make this work locally on an Android device.

1. Audio Interception (The Input)

To process the audio, the app needs continuous access to the sound stream without interrupting the playback of your media app.

- **For Internal Device Audio (Podcasts, Spotify, YouTube):** You should use Android's **AudioPlaybackCapture API** (introduced in Android 10). It allows an app to capture the audio being played by other apps.
 - **How it works:** It requires a **MediaProjection** token (which usually prompts the user to "Start Recording or Casting"). Since it's for personal use, you can grant this permission once when you open your app.
 - **Output:** It provides a raw PCM byte stream of the internal audio, completely ignoring room noise.
- **For External Audio (Watching TV in the living room):** If the goal is to mute the phone (or a connected smart speaker) when the TV plays commercials, use the standard **AudioRecord** API to continuously listen via the device's microphone.

2. The ML Core (Commercial Detection)

Running continuous audio inference on a phone can drain the battery, so the model must be extremely lightweight. Here are two implementation paths:

Approach A: YAMNet / VGGish Embeddings + Logistic Regression (Recommended)

This perfectly matches your idea and is the most efficient route. **YAMNet** and **VGGish** are highly optimized, lightweight audio models created by Google, trained on millions of YouTube audio clips (AudioSet).

1. **Extract the Embeddings:** Instead of using the final classification layer of YAMNet (which predicts 521 generic sound classes), you strip the final layer and extract the penultimate layer. In YAMNet, this gives you a 1024-dimensional embedding vector for every 0.96 seconds of audio.
2. **Train a Logistic Regression Layer:** On your computer, collect a small dataset of "commercials" and "regular content." Run this audio through YAMNet to get the embeddings. Train a simple Logistic Regression model (or a tiny one-layer neural network) on these embeddings to output a binary "Commercial" or "Not Commercial" score.
3. **On-Device Execution:** In your Android app, run the raw audio through the standard TensorFlow Lite YAMNet model to get the 1024-float array. Then, implement the logistic regression directly in Kotlin/Java (just a simple matrix dot-product with your trained weights and a sigmoid activation function).

Approach B: Custom Spectrogram + MobileNet

If YAMNet's embeddings don't capture the subtle nuances of local commercials, you can build an end-to-end custom pipeline.

1. Convert the raw PCM audio stream into a Mel-spectrogram directly in Android (this is just 2D visual representation of audio frequencies).
2. Treat this spectrogram as an image and feed it into a quantized **MobileNetV2** or **MobileNetV3** model running on TensorFlow Lite.
3. MobileNet is highly optimized for Android CPU/GPU delegates and will easily classify the "images" of commercial audio in real-time.

Bonus Heuristic: LUFS / Loudness Spikes

Commercials are notoriously heavily compressed and dynamically flat so they sound louder than the actual show. You can calculate the LUFS (Loudness Units relative to Full Scale) of the PCM buffer. Feeding this single loudness metric into your logistic regression model alongside the YAMNet embeddings will drastically increase your model's accuracy.

3. The Action Module (Muting & Smoothing)

Once the model identifies a commercial, the app needs to manipulate the volume gracefully.

- **Volume Control:** Use Android's `AudioManager` to target `STREAM_MUSIC`. When a commercial triggers, save the current volume integer, and lower the volume to 1 (or 0). When the commercial ends, restore the saved integer.
- **Prediction Smoothing (Debouncing):** You do not want the volume jumping up and down erratically if the ML model misclassifies a single second of audio. You should implement a sliding window or state machine.

- *Example:* Keep a rolling history of the last 5 seconds. The app only triggers the "Mute" action if the logistic regression model outputs >80% probability for 3 consecutive seconds. It only un-mutes when the probability drops below 40% for 4 consecutive seconds.

4. App Architecture Overview

To tie it all together, your Android app would be structured like this:

1. **Foreground Service:** Because modern Android kills background apps aggressively, your app must run as a Foreground Service with a persistent notification (e.g., "Ad Blocker Listening..."). This allows the audio capture to run endlessly while the screen is off.
2. **Audio Thread:** A dedicated background thread pulling PCM data from the `AudioPlaybackCapture` buffer in 1-second chunks.
3. **Inference Pipeline:** The PCM chunk is passed to the TFLite Interpreter (using the NNAPI or CPU delegate). The resulting embedding is multiplied by your custom weights.
4. **State Manager:** Evaluates the rolling average of the predictions and calls `AudioManager` when the threshold is crossed.

Because this is entirely local, the latency will be milliseconds, ensuring the volume drops the moment the commercial's jingle starts.

Sources

1. [Capture video and audio playback | Android media](#)
 2. [Yamnet classification android application - GitHub](#)
-